

A Starter Guide to Building Progressive Web Apps

As the title suggests, this is a primer on building PWAs. The article starts with the basics and takes the reader through various aspects of PWAs, with real life examples.

There are Web applications, native applications and then, there are progressive Web applications (PWAs) that bridge the functionalities of both.

By now, most people would have heard about progressive Web applications on the Internet. However, this term still confuses a few. Those who are recently into Web development from a different domain, or tech enthusiasts like project managers who want to consistently incorporate new technologies into their workflows, are still trying to understand more about this buzzword.

That's why I've decided to clear all the misconceptions around this topic. By the end of this article, hopefully, readers will have a clearer picture about PWA technology, its uses, and how to build one (at a beginner level).

What are progressive Web applications?

Progressive Web applications are built to perform like apps while looking like websites. They have all the functionalities of a native app – those that you use on your smartphone – and still manage to offer the usability of a website.

Ever since Google proposed this concept in 2015, progressive Web application development has been seen as the norm for future Web and app development. With companies looking to optimise the user experience consistently, PWAs fit their requirements in terms of budgets, functionality, development time, and more.



The best of both worlds

This technology offers the best of both worlds. The evolution of browsers and Web development technologies like push APIs, cache and service workers has paved the way for the development of Web applications. These are the online counterparts of native apps that are capable of working offline, receiving push notifications, and delivering an experience that matches that of a native app.

Progressive Web applications, on the other hand, are a step ahead.

- They leverage the potential of a much larger ecosystem to function as a website.
- They eliminate the need for developers to build distinct versions of an app for different operating systems, launching it on multiple app stores and then pushing individual updates.
- Thanks to their functionality as websites, they can be built faster than native or Web apps and be instantly deployed.

The best part of a PWA is that it eliminates the need for backward compatibility — users from any platform or device can run one single code of your website.

This is a win-win for both developers and users. It makes it easier for developers to deploy and maintain the app, and allows users to access all the features of a native app seamlessly.

Why do we need PWAs?

So far, we have only discussed the benefits that progressive Web apps offer. Let me further refine a typical customer's journey when using a native app:

1. Identify the concern one faces.
2. Look for probable solutions online.
3. Realise that an app is the ideal solution.
4. Go to the App Store or Play Store.
5. Search for the app.
6. Get recommendations and ads on the multiple apps.
7. Try out the most recommended one.
8. And then finally decide on one that addresses the original concern.

But this cycle does not end here.

According to statistics, 79 per cent of users abandon an app after a day's use. Besides, 29 per cent of users abandon a downloaded app if they don't find it useful. These statistics aside, there are other factors like the UI of the native app, load time, the amount of space it takes up in storage, offline capabilities, and more, that hinder app usage. Even if just one of the many factors mentioned fails, the app loses a user, triggering the customer cycle again.

So, why progressive Web applications?

Simply because the customer's journey ends at the second step. Once the concern is identified, the user can access the solution very simply. There